Plano de desenvolvimento do indicador I03 (Encontro 10)

SaúdeTech Dashboards · Projeto de Extensão · Novembro de 2026

1. Identificação

Campo	Preenchimento
Grupo	The six
Integrantes	Cailane Soares – 26006303-2 Eloá Santos – 260141149-2 João Pedro Muniz Belizario – 26003438-2 Ketlyn Nicoletti Modesto – 26013912-2; Lucas Henrique Silva Martins – 26005361-2 Pedro Henrique Rosa Andretto – 26007403-2
Indicador escolhido	I03 – Vistorias por tipo de estabelecimento
Pergunta que o indicador responde	Quais tipos de estabelecimento (farmácia, restaurante, clínica, indústria etc.) recebem mais vistorias?
Tipo de gráfico	Barras horizontais, ordenadas da maior para a menor quantidade de vistorias
Repositório (fork)	https://github.com/Laine03/Grupo-da-exten-o.git
Branch de trabalho	feature/dashboard-i03

2. EAP simplificada

Código	Pacote de trabalho	Tarefas	Entregável
1	Preparação	1.1 Fazer fork e clonar; 1.2 Subir o ambiente com Docker; 1.3 Conferir acesso ao banco (usuário somente leitura)	Painel rodando em localhost
2	Dados	2.1 Escrever a consulta SQL (estabelecimentos JOIN vistorias, GROUP BY tipo_atividade); 2.2 Testar no DBeaver e conferir com a contagem total de vistorias; 2.3 Tratar casos sem dados (tipo sem vistoria, resultado vazio)	Consulta validada

Código	Pacote de trabalho	Tarefas	Entregável
3	Interface	3.1 Configurar título e pergunta do cartão; 3.2 Configurar gráfico de barras horizontais com rótulos legíveis; 3.3 Conferir tema claro e escuro	Cartão do indicador I03 no Dashboard
4	Qualidade	4.1 Testar com filtros (período, tipo, agente); 4.2 Testar erro de SQL e resultado vazio; 4.3 Revisar o resultado com o grupo	Checklist de testes
5	Entrega	5.1 Commits no padrão convencional; 5.2 Tag v1.1.0; 5.3 Pull Request	Link do repositório
6	Documentação	6.1 Descrever o indicador no README; 6.2 Salvar este plano em docs/; 6.3 Preparar a apresentação ao professor	README e plano atualizados

3. Cronograma das Sprints

Sprint	Período	Meta	Tarefas da EAP	Entrega ao final	Status
Sprint 1	03/11 a 09/11	Ambiente e dados prontos	1.1 a 1.3, 2.1 a 2.2	Consulta SQL funcionando no DBeaver	Não iniciada
Sprint 2	10/11 a 16/11	Indicador visível no painel	2.3, 3.1 a 3.3	Gráfico aparecendo no Dashboard	Não iniciada
Sprint 3	17/11 a 23/11	Testes e entrega	4.1 a 5.3, 6.1 a 6.3	Pull Request aberto e tag publicada	Não iniciada

Revisão de cada Sprint: ao final de cada Sprint o grupo faz um commit, atualiza a coluna Status e registra aqui o que foi entregue, o que ficou pendente e o que muda na próxima Sprint. Sprint 1: [preencher]. Sprint 2: [preencher]. Sprint 3: [preencher]. (Datas assumidas a partir de 02/11/2026; ajustar ao calendário oficial da disciplina.)

4. Responsabilidades

Tarefa	Responsável (R)	Apoio (A)	Prazo
1. Preparação	Lucas Henrique Silva Martins	Pedro Henrique Rosa Andretto	05/11
2. Dados	João Pedro Muniz Belizario	Cailane Soares	09/11
3. Interface	Eloá Santos	Ketlyn Nicoletti Modesto	16/11
4. Qualidade	Ketlyn Nicoletti Modesto	Cailane Soares	20/11

Tarefa	Responsável (R)	Apoio (A)	Prazo
5. Entrega	Pedro Henrique Rosa Andretto	Lucas Henrique Silva Martins	23/11
6. Documentação	Cailane Soares	Eloá Santos	23/11

Papéis do grupo: Lucas – líder técnico e ambiente; João Pedro – responsável pelos dados (SQL); Eloá – responsável pela interface; Ketlyn – responsável por testes e qualidade; Pedro – responsável por Git, tag e Pull Request; Cailane – responsável pela documentação e apresentação. Todos têm ao menos uma tarefa de código: Cailane escreve o tratamento de resultado vazio (2.3) e os testes de filtros (4.1); Ketlyn apoia a configuração do gráfico (3.2) e escreve os testes (4.1 e 4.2); Lucas e Pedro configuram o ambiente e os commits; João Pedro escreve a consulta; Eloá implementa o cartão.

Consulta prevista (tarefa 2.1): SELECT e.tipo_atividade, COUNT(v.id) AS total_vistorias FROM estabelecimentos e JOIN vistorias v ON v.estabelecimento_id = e.id GROUP BY e.tipo_atividade ORDER BY total_vistorias DESC;

5. Prioridades (MoSCoW)

Prioridade	Entrega	Justificativa
Deve	Consulta SQL correta e gráfico exibido no cartão	Núcleo do indicador: sem isso o I03 não existe
Deve	Commits, tag e Pull Request	Exigido para a avaliação
Deveria	Tratamento de erro e de resultado vazio	Evita derrubar a página e deixa o painel confiável
Poderia	Ajustes de cores e rótulos	Melhora a leitura, mas o indicador funciona sem isso
Não agora	Mapa por bairro, exportação CSV/PDF e comparação entre estabelecimentos	Outros requisitos do dashboard (RF-03, RF-09, RF-10); ficam para versão futura

6. Riscos e plano de resposta

Risco	Probabilidade	Impacto	O que fazer
Se a consulta SQL devolver erro ou resultado vazio, o cartão não exibe o gráfico	Média	Alto	Testar no DBeaver antes de colar no projeto e exibir mensagem no cartão
Se não houver acesso ao banco pela rede, o grupo não consegue testar a consulta	Baixa	Alto	Avisar o professor na hora e usar o banco reproduzido localmente com o seed

Risco	Probabilidade	Impacto	O que fazer
Se um integrante faltar, sua tarefa fica parada	Média	Médio	O apoio (A) da tarefa assume; manter o repositório atualizado
Se houver conflito de merge no Git, o trabalho de alguém pode ser perdido	Média	Médio	Commits pequenos, pull antes de subir e cada pessoa na sua branch
Se o .env ou a senha do banco for enviado ao Git, há vazamento de credenciais	Baixa	Alto	Conferir .gitignore e usar o usuário somente leitura
Se o seed tiver poucos dados (cerca de 100 vistorias), o gráfico pode ficar pouco informativo	Média	Baixo	Explicar o volume na apresentação e testar também com filtros

7. Definição de pronto e controle de versão

O indicador está pronto quando:

- A consulta roda com o usuário somente leitura
- O gráfico aparece no Dashboard nos temas claro e escuro
- Erros de SQL aparecem no cartão sem derrubar a página
- O arquivo `.env` não foi enviado ao Git
- Os commits seguem o padrão `feat`, `fix` ou `docs`
- A tag de versão (ex.: `v1.1.0`) foi publicada e o Pull Request foi aberto

Item	Preenchimento
Branch	feature/dashboard-i03
Commits previstos	feat(dashboard): adiciona consulta do indicador I03; feat(dashboard): configura gráfico de barras por tipo de estabelecimento; fix(dashboard): trata resultado vazio; docs(readme): descreve o indicador I03
Tag prevista	v1.1.0

8. Critérios de avaliação

Critério	Peso	O que se espera
EAP simplificada	20%	Pacotes e tarefas claros, com entregáveis
Cronograma das Sprints	20%	Metas, datas e entregas realistas

Critério	Peso	O que se espera
Responsabilidades	20%	Uma pessoa por tarefa, carga equilibrada
Prioridades	15%	MoSCoW coerente com o prazo
Riscos e definição de pronto	15%	Riscos reais e critérios verificáveis
Clareza e apresentação	10%	Linguagem técnica e domínio do plano

SaúdeTech Dashboards

Indicador I03 — Vistorias por tipo de estabelecimento

Relatório demonstrativo acadêmico. Os dados apresentados neste documento são fictícios e não representam registros oficiais.

1. Contextualização

O projeto SaúdeTech Dashboards propõe organizar informações da Vigilância Sanitária em painéis visuais que apoiem a consulta e a análise pelos servidores. No material do projeto, o indicador I03 é apresentado como "Vistorias por tipo", com visualização em gráfico de barras e foco na comparação entre categorias de estabelecimentos.

2. Objetivo do indicador

Apresentar a quantidade de vistorias realizadas em cada tipo de estabelecimento, permitindo observar a distribuição da atividade fiscalizatória e identificar categorias que concentram maior volume de vistorias.

3. Dados e modelo demonstrativo

Para construir uma demonstração executável, foi criado um schema PostgreSQL isolado chamado demo_saudetech_i03, com as tabelas estabelecimentos, agentes e vistorias. Os registros foram inventados para fins acadêmicos. O modelo deve ser ajustado ao esquema oficial antes de qualquer integração.

4. Consulta SQL principal

```
SELECT
    e.tipo_estabelecimento,
    COUNT(v.id) AS total_vistorias
FROM estabelecimentos e
INNER JOIN vistorias v ON v.estabelecimento_id = e.id
GROUP BY e.tipo_estabelecimento
ORDER BY total_vistorias DESC;
```

A consulta relaciona cada vistoria ao estabelecimento correspondente, agrupa os registros pelo tipo e conta quantas vistorias existem em cada categoria.

5. Visualizações

O dashboard HTML contém: (a) gráfico de barras com total de vistorias por tipo; (b) evolução mensal demonstrativa; (c) cartões com total de vistorias, quantidade de categorias e quantidade de registros não conformes; (d) filtro por tipo; e (e) tabela-resumo.

6. Como executar

1. Executar o arquivo saudetech_i03_demo.sql em uma conexão PostgreSQL de teste pelo DBeaver.
2. Conferir a consulta final de validação da carga demonstrativa.
3. Abrir dashboard_i03_demo.html em um navegador.
4. Para integração real, mapear as tabelas e colunas oficiais e conectar a camada visual à consulta do banco.

7. Limitações e próximos passos

A demonstração não está conectada ao banco oficial, não utiliza dados reais e não comprova implantação. Como próximos passos, recomenda-se validar o modelo com o responsável pelo banco, adaptar a consulta aos nomes reais, definir filtros de período e atualizar o dashboard a partir de uma fonte de dados autorizada.

8. Considerações finais

O protótipo demonstra como o indicador I03 pode ser calculado e apresentado de forma visual. A separação entre dados fictícios e dados oficiais mantém a demonstração transparente e permite evoluir o trabalho para uma integração validada.

SaúdeTech Dashboards — Indicador I03

O que é

Protótipo acadêmico demonstrativo do indicador **Vistorias por tipo de estabelecimento**, apresentado no material do projeto como gráfico de barras para comparar categorias e apoiar a identificação de grupos com maior concentração de ocorrências.

Conteúdo

- `saudetech_i03_demo.sql` : cria um schema isolado no PostgreSQL, popula dados fictícios e apresenta consultas SQL.
- `dashboard_i03_demo.html` : dashboard demonstrativo com gráfico de barras, evolução mensal, indicadores-resumo, tabela e filtro por tipo.
- `Relatorio_I03.docx` : documentação do objetivo, método, consultas e limitações.
- `Apresentacao_I03.pptx` : roteiro de apresentação de aproximadamente 5 minutos.

Como executar o SQL no DBeaver

1. Abra o DBeaver e conecte-se a um banco PostgreSQL de teste.
2. Abra um editor SQL (botão direito na conexão → SQL Editor → New SQL Script).
3. Abra ou cole o conteúdo de `saudetech_i03_demo.sql` .
4. Execute o script completo.
5. Confira a última consulta: ela deve retornar 15 estabelecimentos, 4 agentes e 40 vistorias.
6. Execute separadamente a consulta principal e use o resultado para criar um gráfico de barras no DBeaver, se desejar.

O script remove e recria somente o schema `demo_saudetech_i03` . Não execute em ambiente de produção.

Como abrir o dashboard

Abra `dashboard_i03_demo.html` em um navegador (Chrome, Edge ou Firefox). Não precisa instalar bibliotecas ou estar conectado à internet.

Como publicar no GitHub

1. Crie ou abra o repositório do grupo.
2. Faça upload destes arquivos e, se desejar, crie as pastas `sql/` , `dashboard/` e `docs/` .

3. Use uma mensagem de commit clara, por exemplo: `feat: adiciona prototipo demonstrativo do indicador I03`.
4. Não inclua senhas, tokens ou credenciais do banco no repositório.

Limitações importantes

- Os registros e valores deste pacote são **fictícios**, criados exclusivamente para demonstração.
- O esquema SQL é independente e ilustrativo. Embora use os conceitos de estabelecimentos e vistorias descritos no material, os nomes e campos precisam ser comparados com o banco oficial antes de integrar.
- O HTML usa um conjunto demonstrativo próprio; não consulta automaticamente o PostgreSQL.
- Portanto, este pacote demonstra a lógica do indicador e a apresentação visual, mas não comprova integração com o sistema oficial nem resultados reais da Vigilância Sanitária.

```
-- SaúdeTech Dashboards | Indicador I03
-- Demonstração acadêmica com dados FICTÍCIOS.
-- Este script cria um schema isolado e não altera tabelas reais do projeto.
-- Banco: PostgreSQL | Execução sugerida: DBeaver
--
-- Indicador: Vistorias por tipo de estabelecimento
-- Gráfico principal: barras (quantidade de vistorias por tipo).
--
-- ATENÇÃO: nomes de tabelas/campos e registros abaixo formam um modelo
-- demonstrativo independente. Ajustar ao banco oficial antes de integrar.

DROP SCHEMA IF EXISTS demo_saudetech_i03 CASCADE;
CREATE SCHEMA demo_saudetech_i03;
SET search_path TO demo_saudetech_i03;

CREATE TABLE estabelecimentos (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(120) NOT NULL,
    tipo_estabelecimento VARCHAR(60) NOT NULL,
    bairro VARCHAR(80) NOT NULL
);

CREATE TABLE agentes (
    id SERIAL PRIMARY KEY,
    nome VARCHAR(120) NOT NULL
);

CREATE TABLE vistorias (
    id SERIAL PRIMARY KEY,
    estabelecimento_id INTEGER NOT NULL REFERENCES estabelecimentos(id),
    agente_id INTEGER NOT NULL REFERENCES agentes(id),
    data_vistoria DATE NOT NULL,
    resultado VARCHAR(30) NOT NULL CHECK (
        resultado IN ('Conforme', 'Não conforme', 'Parcialmente conforme')
    ),
    observacoes TEXT
);

INSERT INTO estabelecimentos (nome, tipo_estabelecimento, bairro) VALUES
('Restaurante Sabor Caseiro','Restaurante','Centro'),
('Restaurante Bom Prato','Restaurante','Gleba Palhano'),
('Lanchonete Avenida','Restaurante','Vila Casoni'),
('Farmácia Saúde','Farmácia','Centro'),
('Drogaria Vida','Farmácia','Jardim Leonor'),
('Clínica Bem Estar','Clínica','Centro'),
('Clínica Integrar','Clínica','Jardim Quebec'),
('Mercado União','Mercado','Vila Nova'),
('Mercado Popular','Mercado','Centro'),
('Padaria Pão Quente','Padaria','Jardim Bandeirantes'),
('Padaria Trigo Bom','Padaria','Centro'),
('Laboratório Vida','Laboratório','Centro'),
('Indústria Alimentos Norte','Indústria','Parque Industrial'),
('Salão Beleza & Cia','Serviços de beleza','Vila Brasil'),
('Hotel Central','Hotel','Centro');

INSERT INTO agentes (nome) VALUES
```

```
('Agente Ana'), ('Agente Bruno'), ('Agente Carla'), ('Agente Diego');

-- Registros de exemplo fictícios, distribuídos por tipo e por mês.
INSERT INTO vistorias (estabelecimento_id, agente_id, data_vistoria, resultado, observacoes) VALUES
(1,1,'2026-01-08','Não conforme','Ausência de identificação de validade'),
(2,2,'2026-01-12','Conforme',''),
(3,3,'2026-01-19','Parcialmente conforme','Ajuste de armazenamento'),
(4,1,'2026-01-22','Conforme',''),
(5,4,'2026-01-28','Conforme',''),
(6,2,'2026-02-03','Não conforme','Documentação incompleta'),
(7,3,'2026-02-09','Conforme',''),
(8,4,'2026-02-14','Não conforme','Temperatura inadequada'),
(9,1,'2026-02-18','Conforme',''),
(10,2,'2026-02-20','Parcialmente conforme','Higienização a melhorar'),
(11,3,'2026-02-25','Conforme',''),
(12,4,'2026-03-02','Conforme',''),
(13,1,'2026-03-05','Não conforme','Armazenamento irregular'),
(14,2,'2026-03-10','Conforme',''),
(15,3,'2026-03-13','Conforme',''),
(1,4,'2026-03-18','Não conforme','Reincidência em item sanitário'),
(2,1,'2026-03-21','Conforme',''),
(4,2,'2026-03-24','Conforme',''),
(5,3,'2026-04-02','Parcialmente conforme','Adequação de registros'),
(6,4,'2026-04-06','Conforme',''),
(7,1,'2026-04-11','Não conforme','Pendência documental'),
(8,2,'2026-04-15','Conforme',''),
(9,3,'2026-04-19','Conforme',''),
(10,4,'2026-04-22','Não conforme','Controle de pragas'),
(11,1,'2026-04-26','Conforme',''),
(12,2,'2026-05-03','Conforme',''),
(13,3,'2026-05-08','Não conforme','Identificação de produtos'),
(14,4,'2026-05-12','Conforme',''),
(15,1,'2026-05-16','Parcialmente conforme','Plano de limpeza'),
(3,2,'2026-05-20','Conforme',''),
(1,3,'2026-06-01','Conforme',''),
(2,4,'2026-06-04','Não conforme','Falha em procedimento'),
(4,1,'2026-06-08','Conforme',''),
(5,2,'2026-06-12','Conforme',''),
(6,3,'2026-06-16','Parcialmente conforme','Atualização de documentos'),
(8,4,'2026-06-20','Conforme',''),
(10,1,'2026-06-24','Conforme',''),
(13,2,'2026-06-27','Não conforme','Condições de armazenamento'),
(14,3,'2026-07-02','Conforme',''),
(15,4,'2026-07-06','Conforme','');

-- CONSULTA PRINCIPAL (I03): quantidade de vistorias por tipo.
SELECT
    e.tipo_estabelecimento,
    COUNT(v.id) AS total_vistorias
FROM estabelecimentos e
INNER JOIN vistorias v ON v.estabelecimento_id = e.id
GROUP BY e.tipo_estabelecimento
ORDER BY total_vistorias DESC, e.tipo_estabelecimento;

-- CONSULTA COMPLEMENTAR: vistorias e não conformidades por tipo.
SELECT
```

```
    e.tipo_estabelecimento,
    COUNT(v.id) AS total_vistorias,
    COUNT(*) FILTER (WHERE v.resultado = 'Não conforme') AS nao_conformes,
    COUNT(*) FILTER (WHERE v.resultado = 'Parcialmente conforme') AS parcialmente_conformes,
    ROUND(
        100.0 * COUNT(*) FILTER (WHERE v.resultado = 'Não conforme')
        / NULLIF(COUNT(v.id), 0), 1
    ) AS percentual_nao_conforme
FROM estabelecimentos e
INNER JOIN vistorias v ON v.estabelecimento_id = e.id
GROUP BY e.tipo_estabelecimento
ORDER BY total_vistorias DESC;

-- CONSULTA PARA LINHA DO TEMPO: total de vistorias por mês.
SELECT
    DATE_TRUNC('month', data_vistoria)::date AS mes,
    COUNT(*) AS total_vistorias
FROM vistorias
GROUP BY DATE_TRUNC('month', data_vistoria)
ORDER BY mes;

-- CONSULTA COM FILTRO DE PERÍODO (edite as datas conforme necessário).
SELECT
    e.tipo_estabelecimento,
    COUNT(v.id) AS total_vistorias
FROM estabelecimentos e
JOIN vistorias v ON v.estabelecimento_id = e.id
WHERE v.data_vistoria BETWEEN DATE '2026-01-01' AND DATE '2026-06-30'
GROUP BY e.tipo_estabelecimento
ORDER BY total_vistorias DESC;

-- Conferência rápida da carga demonstrativa.
SELECT
    (SELECT COUNT(*) FROM estabelecimentos) AS estabelecimentos_cadastrados,
    (SELECT COUNT(*) FROM agentes) AS agentes_cadastrados,
    (SELECT COUNT(*) FROM vistorias) AS vistorias_registradas;
```

```
-- Consulta principal do indicador I03
SET search_path TO demo_saudetech_i03;
SELECT e.tipo_estabelecimento, COUNT(v.id) AS total_vistorias
FROM estabelecimentos e
JOIN vistorias v ON v.estabelecimento_id = e.id
GROUP BY e.tipo_estabelecimento
ORDER BY total_vistorias DESC;
```

SAÚDETECH DASHBOARDS • VIGILÂNCIA SANITÁRIA

Indicador I03 — Vistorias por tipo de estabelecimento

Visão demonstrativa para análise da distribuição das vistorias.

Protótipo acadêmico: todos os números exibidos são fictícios e servem apenas para demonstrar o funcionamento. Não representam dados oficiais da VISA.

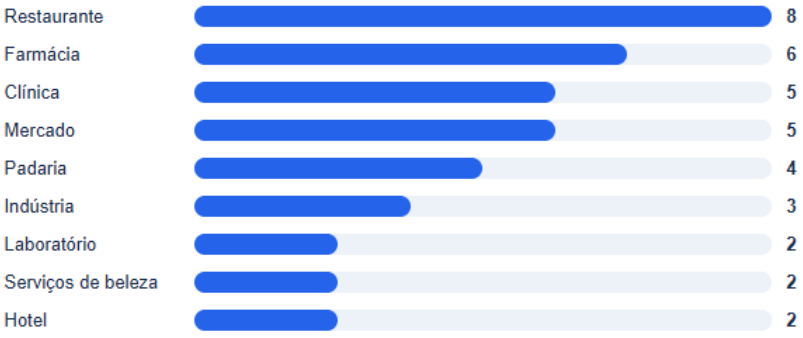
Total de vistorias demonstrativas	Tipos de estabelecimento	Vistorias não conformes
37	9	11

Tipo de estabelecimento: Todos

Os gráficos e a tabela são atualizados pelo filtro.

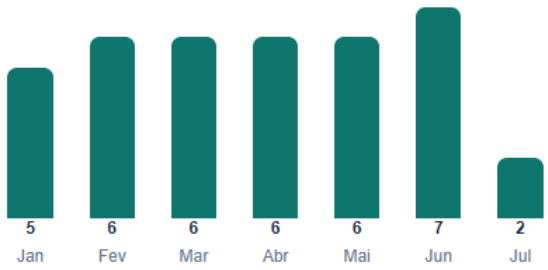
Vistorias por tipo

Gráfico de barras — quantidade de vistorias por categoria.



Evolução mensal

Quantidade de vistorias registradas por mês (amostra).



Resumo por categoria

Detalhamento complementar para apoiar a leitura do indicador.

Tipo	Vistorias	Não conformes	Parcialmente conformes
Restaurante	8	3	1
Farmácia	6	1	1
Clínica	5	2	1
Mercado	5	1	0
Padaria	4	1	1
Laboratório	2	0	0
Indústria	3	3	0
Serviços de beleza	2	0	0
Hotel	2	0	1